

1 Программная модель Triton

CUDA предоставляет программисту достаточно низкоуровневую модель: потоки, блоки, работу с SMEM и RMEM. Это удобно, когда нужен полный контроль над поведением программы, но писать кернелы с производительностью на уровне cuBLAS крайне сложно.

При работе с нейросетями и, например, тестировании новых архитектур, хотелось бы работать на более высоком уровне.

Обычно мы не хотим вручную описывать поведение каждого потока. Вместо этого проще думать о блоках данных: фрагментах векторов, тайлах матриц и так далее.

Именно эту идею использует **тритон (Triton)** — Python-подобный **DSL (Domain Specific Language)**. Пользователь описывает вычисления на уровне блоков данных, а Triton затем компилирует этот код в низкоуровневое представление для выполнения на GPU.

1.1 Зачем нужен Triton?

На данный момент мы знаем только два инструмента для работы с нейронными сетями на GPU.

Первый вариант — вызывать готовые кернелы на `PyTorch`. Это одна крайность: удобно, но иногда это может быть малоэффективно, поскольку без слияния кернелов это может значительно замедлить работу модели и раздуть активации.

Второй вариант — писать собственные CUDA-кернелы. Это другая крайность: мы можем достичь максимальной эффективности, но это потребует огромных сил.

Тритон занимает промежуточное место: мы можем реализовывать собственные кернелы малыми силами. В некотором смысле использование тритон напоминает закон Парето: 20% усилий дают 80% результата.

1.2 Кернел в Triton

Тритон-кернел аналогичен кернелу в CUDA. Это функция, которая будет исполняться на GPU. Такую функцию мы помечаем декоратором `@triton.jit`.

Синтаксис этой функции похож на Python, но она не выполняется интерпретатором Python построчно, а работает иначе. Тритон-кернел при запуске сначала проходит через компилятор. Код компилируется не заранее, а во время первого запуска.

Когда мы впервые вызываем Тритон-кернел, Тритон анализирует функцию, строит промежуточное представление программы, оптимизирует его и генерирует низкоуровневый код для GPU. Такой подход называется **just-in-time (JIT)** компиляцией.

После компиляции результат обычно кэшируется и переиспользуется во время следующих запусков. Поэтому первый запуск кернела может быть значительно медленнее последующих.

Запуск тритон-кernels похож на запуск CUDA-кernels. Однако в отличие от CUDA, где мы задавали сетку блоков и потоков, в тритон мы задаем сетку из **program instance** — экземпляров программы.

1.3 Program Instance

В CUDA основная исполнительная единица — поток. Основная единица исполнения в Тритон — **экземпляр программы (program instance)**. Этот экземпляр программы отвечает не за один элемент, а за целый блок данных: фрагмент вектора, строку матрицы, тайл матрицы и так далее. Размер этого блока задается с помощью `BLOCK_SIZE`.

Чтобы понять, какой блок данных должен обработать текущий экземпляр программы, используется `pid` — идентификатор экземпляра программы.

Далее нужно построить глобальные индексы элементов, которые относятся к текущему блоку

$$\text{offset}_j = \text{pid} \cdot \text{BLOCK_SIZE} + \text{local_id}_j,$$

где `local_id` — это локальные индексы внутри блока. Их можно получить с помощью `tl.arange(0, BLOCK_SIZE)`.

Если `BLOCK_SIZE = 4`, то для разных значений `pid` получим

```
pid = 0: offset = (0  1  2  3),
pid = 1: offset = (4  5  6  7),
pid = 2: offset = (8  9 10 11),
```

...

Далее вычисления внутри kernels выполняются сразу для всего блока.

Например, сложение векторов будет выглядеть следующим образом:

```
x = tl.load(x_ptr + offsets)
y = tl.load(y_ptr + offsets)

z = x + y

tl.store(z_ptr + offsets, z)
```

Один экземпляр программы вычисляет сразу блок итогового вектора, в отличие от CUDA, где один поток вычислял бы один элемент. Экземпляр программы тритона — это более крупная логическая единица. При выполнении, компилятор сам отобразит операции внутри экземпляра программы на более низкоуровневое выполнение на GPU.

Иными словами, Тритон использует блочную векторизованную модель программирования. По форме она похожа на **single instructions multiple**

data (SIMD) модель: одна программа применяется для блока данных. Но по итогу она исполняется на SIMT-подобной архитектуре GPU.

Как и в CUDA, в Тритон мы можем задать несколько осей при задании сетки. Тогда по каждой из осей нужно будет вычислить свой `pid`.

2 Профилирование

Оптимизация без явных измерений и оценок каких то метрик превращается в угадывание. Даже если некоторая оптимизация кажется очевидно, она может не ускорить программу.

Например, мы можем ускорить один кернел, но весь шаг обучения при этом остается таким же медленным, потому что узкое место на самом деле находится, например, в загрузке данных и синхронизации.

Поэтому любая оптимизация должна проверяться профилированием.

Профилирование можно разделить на несколько уровней: бечмарк, профилирование отдельных кернелов и профилирование всего шага обучения.

2.1 Бенчмарк

Бенчмарк — это измерение времени выполнения операций, по отдельности или вместе.

Ранее, в контексте стримов, мы говорили о том, что кернелы выполняются асинхронно. Это значит, что когда мы вызываем GPU-операцию, CPU обычно не ждет её завершения. Поэтому если мы в CPU-коде решим замерить выполнение программы на GPU, то мы получим не то что хотим, ведь таким образом мы замеряем именно время инициализации кернела на CPU.

CPU ставит операцию в очередь стрима. Поэтому, чтобы измерить время выполнения этой операции, необходимо до и после неё поставить CUDA-события. Далее мы можем оценить время между срабатыванием обоих событий.

Нельзя делать выводы по первому запуску. Первые итерации часто медленные: может происходить инициализация CUDA Runtime, JIT компиляции, аллокации буферов памяти, работа с кэшами и так далее.

Поэтому перед измерениями обычно делают несколько разогревочных запусков (**warmup-запусков**).

Оценивать время работы по одному замеру тоже ненадежно: время может колебаться из-за фоновых процессов, загрузки CPU, температуры GPU и так далее. Поэтому операцию запускают много раз и смотрят не одно число, а статистику: медиану и, например 25%, 75% перцентили (иногда еще и 90% перцентили).

Важно различать бенчмарк одной отдельной операции и полный замер всего шага обучения. Ускорение в первом случае может быть почти не заметно на всём масштабе: если операция занимала малую долю времени, то её оптимизация почти ничего не изменит.

Это явление описывает закон Амдала:

$$S = \frac{1}{(1-p) + \frac{p}{s}},$$

где p — доля оптимизированной части, а s — ускорение этой части.

Например, если операция занимала 10% времени шага, а мы ускорили её в 10 раз (что на самом деле очень хороший результат), то общее ускорение будет равно:

$$S = \frac{1}{0.9 + 0.01} \approx 1.1,$$

то есть примерно 10%.

2.2 Профилирование кернелов

Бенчмарк показывает сколько времени заняло выполнение операции. Однако не объясняет, почему операция заняла именно столько времени. Для этого нужно смотреть, сколько времени занимают те или иные составляющие самого кернела.

Для профилирования кернелов используются такие инструменты как **Nsight Compute**. Такие профилировщики позволяют смотреть, как кернел использует вычислительные блоки, тензорные ядра и памяти (а **Nsight Compute** также позволяет построить диаграмму обращений к разным уровням памяти).

Если кернел — `compute-bound`, то важно смотреть на достигнутое значение `FLOP/s`.

Если же ситуация иная, и кернел `memory-bound`, то полезно оценивать пропускную способность памяти.

2.3 Профилирование обучения

Отдельный кернел может быть быстрым, но весь шаг обучения всё равно может оставаться медленным. Поэтому также важно профилировать не только кернелы, но и всю итерацию обучения. Или по крайней мере основные шаги: прямое распространение, вычисление лосса, обратное распространение и шаг оптимизатора.

Также, важно учитывать работу всех стримов, поскольку задержка на одном стриме может блокировать все остальные стримы.

Для такого анализа используются таймлайн-профилировщики: `torch.profiler` и **Nsight Systems**. Они показывают не только отдельные операции, но и их расположение во времени: когда работает хост, когда работает девайс, где на таймлайне девайса находится запускаемый на таймлайне хоста кернел, какие стримы используются, где есть пустые промежутки (когда девайс простаивает) и так далее.

Для распределенного обучения также важно смотреть на то, как хорошо стрим коммуникаций перекрывается стримом вычислений, но об этом мы поговорим в следующем разделе.